
Zero-Shot Goal Recognition with Large Language Models

Kin Max Piamolini Gusmão
PUCRS
Porto Alegre, Brazil
kin.gusmao@edu.pucrs.br

Nathan Gavenski
King’s College London
London, England
nathan.schneider_gavenski@kcl.ac.uk

Nir Oren
University of Aberdeen
Aberdeen, Scotland
n.oren@abdn.ac.uk

Felipe Meneguzzi
University of Aberdeen / PUCRS
Aberdeen, Scotland
felipe.meneguzzi@abdn.ac.uk

Abstract

Large language models have recently reached near-parity with classical planners on well-known planning domains, yet this competence relies on world-knowledge exploitation rather than genuine symbolic reasoning. Goal recognition is a complementary abductive task structurally better suited to LLM strengths: it consists of evaluating consistency with world knowledge rather than generating novel action sequences. This paper provides the first systematic zero-shot evaluation of frontier LLMs as goal recognisers on key classical PDDL benchmarks. Our results show that LLM competence on goal recognition is uneven: some models scale with evidence and approach landmark-based accuracy at full observations, while others remain anchored to world-knowledge priors regardless of how much evidence accumulates. Qualitative analysis of model reasoning traces reveals that this divergence reflects a fundamental difference in evidence integration rather than domain familiarity. These findings position goal recognition as a principled benchmark for the foundational planning knowledge of LLMs.

1 Introduction

Large language models (LLMs) have recently been adopted as foundational components in a wide range of applications, including those that require reasoning. Recent benchmarking shows that these models, including reasoning models, have reached near-parity with classical planners on well-known planning domains [21, 3]. This competence, however, rests on a brittle foundation since performance collapses when domain and object names are obfuscated, revealing that LLMs exploit world knowledge rather than performing genuine symbolic reasoning [18, 20].

Goal and plan recognition are related but fundamentally different tasks [13, 14, 8]. Given a set of candidate goals and a partial sequence of an agent’s observed actions, the task is to infer which goal the agent pursues, alternatively, what plan it uses to achieve such goal. Unlike planning, goal recognition is abductive: the recogniser does not construct a valid action sequence but instead asks which hypothesis best explains the evidence so far. Evaluating partial evidence against a fixed hypothesis set aligns naturally with how LLMs leverage the world knowledge encoded in their training, making goal recognition a principled benchmark for probing that foundational planning knowledge. A model that relies solely on world-knowledge priors will produce stable predictions regardless of how much evidence accumulates, revealing the floor set by domain familiarity; a model that scales with observations demonstrates capabilities beyond that floor.

This paper investigates whether LLM competence on known planning domains transfers to goal recognition. We run the first systematic zero-shot evaluation of frontier LLMs as goal recognisers on classical PDDL benchmarks, comparing them against the landmark-based approach of Pereira et al. [11]. We evaluate on four domains chosen for their contrasting characteristics (Section 4): *blocks-world*, *campus*, *driverlog*, and *dock-worker robots*. Our main contributions are: (i) The first systematic zero-shot evaluation of frontier LLMs on classical PDDL goal recognition benchmarks, comparing against landmark-based goal recognition [11]. (ii) Identification of three qualitatively distinct LLM behaviour patterns: GPT-4o achieves competitive recall at low observation counts but never improves with more evidence; GPT-OSS and GPT-5.4 scale with observations, approaching landmark-based accuracy at full observations; and Qwen 3.5 fails almost entirely on all but the simplest domain. (iii) Evidence that high spread is a shared failure mode for LLMs at sparse observations, and that accumulating evidence is not a solution for all models. (iv) Our results provide a valuable extension to the insights from the mystery-domain findings of Valmeekam et al. [18, 20]. Even on familiar domains with standard names, goal recognition shows that frontier LLM performance deteriorates as plans and the space of hypothesis grow larger, suggesting that the bottleneck is not whether LLMs know the domain but whether they can integrate a growing sequence of observations into a coherent prediction. (v) A qualitative analysis of model reasoning traces that identifies shared failure modes (confabulation, overconfidence at sparse observations, position bias, and errors at high observability) and derives concrete interventions to guide the development of LLM-based goal recognisers.

2 Background

Goal recognition is the problem of inferring the goal of an observed agent from a partial sequence of its actions. This task is intimately related to that of planning, which consists of determining how to achieve a given goal G from a known initial state $\mathcal{I}$. However, instead of having to simulate paths from $\mathcal{I}$ towards G , goal recognition is an inverse problem, that of inferring the true goal G^* of the observed agent from observed actions. Ramírez and Geffner [2009, 2010] formalise this problem in terms of classical planning, and Mirsky et al. [8] survey its breadth.

Definition 1 (Goal Recognition Problem). *A **goal recognition problem** is a tuple $\mathcal{P}_{GR} = \langle \Xi, \mathcal{I}, \mathcal{G}, \mathcal{O} \rangle$, where Ξ is a planning domain, $\mathcal{I}$ is the initial state, $\mathcal{G} = \{G_1, \dots, G_n\}$ is a finite set of candidate goal hypotheses, and $\mathcal{O} = \langle a_1, \dots, a_k \rangle$ is an ordered sequence of observed actions. The true goal $G^* \in \mathcal{G}$ is unknown.*

Given a goal recognition problem, the task is to return a ranking or probability distribution over $\mathcal{G}$ given $\mathcal{O}$, identifying G^* as the most likely candidate. In practice, many approaches return the probability distribution over all $G \in \mathcal{G}$ rather than a strict ordering, and allow for ties. We note that the notion of *partial observability* in goal recognition differs from partially observable Markov decision processes [7]. The recogniser never observes the world state directly; it receives only an ordered subset of the actions the agent executes, and must infer the goal from that incomplete trace.

Definition 2 (Observation). *Let $\pi = \langle a_1, \dots, a_m \rangle$ be the full plan executed by the observed agent. An **observation sequence** is a tuple $\mathcal{O} = \langle a_{i_1}, \dots, a_{i_k} \rangle$ where $1 \leq i_1 < i_2 < \dots < i_k \leq m$, that is, an ordered subsequence of π that preserves action order but may omit actions at any position. The **observation percentage** is $100 \cdot k/m$.*

Because $\mathcal{O}$ is a subsequence rather than a prefix, the recogniser has no information about actions at unobserved positions. Observations consist of ground actions rather than state snapshots; this is the standard assumption in classical goal recognition [13, 7]. We assume all observed actions are accurate, i.e., that the partial observation is an ordered subset of π . Noisy observations, in which some observed actions are replaced by incorrect ones, form a distinct problem setting that we explicitly exclude. At low observation percentages the evidence is sparse and many candidate goals remain consistent with $\mathcal{O}$, and as the recogniser receives more actions, disambiguation becomes easier.

Pereira et al. [11] develop an efficient approach to goal recognition based on planning *landmarks*. A landmark for a goal G is an action that any agent pursuing G must eventually execute, regardless of which specific plan it follows; the set of all such actions is $L(G)$. Computing $L(G)$ is expensive, in the worst case as hard as planning itself [4], but it can be approximated in polynomial time via delete-relaxed landmark extraction and done offline once per problem. The resulting landmark sets encode knowledge about each goal analogously to how an LLM’s parameters encode world knowledge, i.e., a costly upfront computation that makes subsequent inference cheap.

Given an observation sequence $\mathcal{O}$, recognition then proceeds by counting which landmarks each candidate goal has already achieved. The landmark score of G is the fraction of its landmarks observed so far $\text{score}(G, \mathcal{O}) = |\{l \in L(G) : l \in \mathcal{O}\}| / |L(G)|$. The most-likely goal set is $\hat{\mathcal{G}} = \arg\max_{G \in \mathcal{G}} \text{score}(G, \mathcal{O})$. Because landmark achievement accumulates monotonically as $\mathcal{O}$ grows, $\text{score}(\hat{G}^*, \mathcal{O})$ tends to increase relative to false goals with each additional observation, so recognition accuracy improves steadily as the observation percentage rises. This monotonic scaling behaviour is the key property against which we compare LLM-based recognisers in Section 5.

3 Zero-Shot Goal Recognition with Large Language Models

A recogniser requires world knowledge of planning domains for zero-shot goal recognition, as well as the ability to update its goal assessments as new observations arrive, a demand that pattern matching alone cannot satisfy. This section introduces our approach to zero-shot goal recognition using LLMs. We first describe our prompting strategy, then situate goal recognition within the complexity landscape of planning to motivate it as a natural benchmark for the foundational planning knowledge of LLMs.

3.1 Prompting Strategy

We treat zero-shot goal recognition as a structured prompting task. We provide no fine-tuning, few-shot examples, or domain-specific context. The evaluation is strictly zero-shot to isolate the contribution of the LLM’s world knowledge.

The prompt has four parts. The first part assigns the model a role as an expert in AI planning and describes each input it will receive: (i) the PDDL domain with its action schemas (names, typed parameters, preconditions, and effects); (ii) the problem template with the list of objects, the initial state, and a placeholder for the goal; (iii) the candidate goal set $\mathcal{G}$, where each $G \in \mathcal{G}$ is a conjunction of facts; and (iv) the observation sequence $\mathcal{O}$ as an ordered list of grounded actions. The second part states the task: for each $G \in \mathcal{G}$, assign a likelihood score in $[0, 1]$ such that the scores sum to one, then return the set $\hat{\mathcal{G}}$ of goals tied at the highest score, together with a free-text explanation of the reasoning. The third part inserts the concrete problem instance. The fourth part provides a rigid output format template, listing each hypothesis with its score, the most-likely goal set $\hat{\mathcal{G}}$, and the reasoning trace, and instructs the model to follow it exactly, without additional text or formatting. This explicit format constraint allows us to parse the model’s output programmatically without post-hoc cleaning. Figure 2 in Appendix A shows the full template.

The normalised scoring requirement follows directly from the formalisation of goal recognition in Section 2. The standard output of a recogniser is a probability distribution over $\mathcal{G}$ [13, 14], and requiring scores to sum to one operationalises this within the prompt. The free-text explanation serves one purpose: it provides interpretable evidence of the model’s decision process for the qualitative analysis in Section 6. Because the output format places the reasoning block *after* the scores and the predicted goal set, the explanation is a post-hoc rationale rather than a chain-of-thought that guides the likelihood assignments [22]. We adopt the standard correctness criterion of Ramírez and Geffner [13], Ramírez and Geffner [14], Pereira et al. [11] whereby a recogniser is correct on a problem when the true goal G^* is contained in the returned set $\hat{\mathcal{G}}$. This is the same criterion from the landmark-based baseline against which we compare.

3.2 Goal Recognition as an LLM Benchmark

Ramírez and Geffner [13], Ramírez and Geffner [14] formalise goal recognition as a compilation to planning: for each candidate goal $G \in \mathcal{G}$, two planning problems are solved to determine whether the observations $\mathcal{O}$ are consistent with an optimal plan for G . Since classical propositional planning is PSPACE-complete [1] and this compilation preserves that hardness [9], goal recognition via planning is no easier than planning itself. Pereira et al. [11] avoid the compilation entirely, and separate the complexity of understanding the domain dynamics from that of incorporating observation evidence. First, their approach extracts landmarks for each goal hypothesis ($L(G)$). Then, it scores each candidate goal by comparing observations with the precomputed landmarks. This yields a complexity of $O(|\mathcal{G}| \times |L(G)| \times |\mathcal{O}|)$, where $|L(G)|$ is bounded by the number of ground actions in the domain. LLM-based recognition requires a single forward pass over the prompt, with cost dominated by the size of the input rather than the planning state space.

The key structural distinction is between *generative* and *discriminative* tasks. Planning asks a model to *produce* a valid action sequence from scratch, requiring consistent long-horizon causal reasoning. Goal recognition asks a model to *evaluate* whether an observed sequence $\mathcal{O}$ is consistent with each candidate goal $G \in \mathcal{G}$, a discriminative task. Given the same PDDL representation, goal recognition never requires generating actions, since the model already observed the agent’s behaviour, and the model’s role is to interpret it. This distinction suggests that LLMs should be structurally better suited to goal recognition than to planning, since evaluating consistency with world knowledge is closer to what LLMs reliably do well [18, 20].

Beyond its practical interest, goal recognition constitutes a principled benchmark for the foundational planning knowledge encoded in LLMs. A model with strong domain knowledge will have prior beliefs about which goals are likely that suffice at low observability, where little evidence is available to discriminate between hypotheses. A model relying solely on such priors, however, will produce approximately constant predictions regardless of how much evidence accumulates; correct recognition at higher observability additionally requires updating those beliefs incrementally as each new action is observed, a capability that world knowledge alone cannot provide. We therefore hypothesise that LLMs will perform competitively against landmark-based recognition at low observation rates, where foundational knowledge priors suffice, but that only models capable of integrating new evidence will improve further. The resulting divergence (a world-knowledge floor at sparse observations and an evidence-dependent ceiling thereafter) is the central signal our experiments are designed to measure, and is what distinguishes world-knowledge retrieval from genuine evidential reasoning.

4 Experimental Setup

To understand the capabilities of LLMs in solving goal recognition problems, we compare them against a state-of-the-art symbolic approach, namely the landmark-based method by Pereira et al. [11]. For the LLMs, we select two commercial and two open-weight models. Among commercial models, we choose OpenAI’s GPT-4o, for offering a balanced trade-off between cost and performance, and GPT-5.4, the frontier OpenAI model at the time of our experiments, for representing the upper bound of available proprietary capability. Among open-weight models, we choose GPT-OSS-20B, the open-weight variant of the GPT line¹, enabling a direct comparison between open and closed models within the same training lineage, and Qwen 3.5-9B², for representing a competitive open-weight model from a distinct model family and organisation, deployable locally without API dependency. We run the experiments on a machine equipped with a 2GHz Intel Xeon Gold 6138 CPU with 32 GB of RAM and an NVIDIA RTX 2080 Ti GPU with 11 GB of VRAM.

We draw problems from the goal and plan recognition benchmark of Pereira and Meneguzzi [10], which comprises 15 PDDL domains. Due to the cost of running commercial LLM APIs at scale, we evaluate on four domains due to their varied characteristics: *Blocks World*, *Campus*, *Driverlog*, and *Dock-Worker Robots (DWR)*. *Blocks World* is a classic combinatorial domain with a vocabulary well-represented in LLM training data; failure here cannot be attributed to an unfamiliar vocabulary, making it the primary test of evidence-integration ability. *Campus* is a simple control domain with short plans and a small goal space; it serves as a minimum competence check, since any approach that fails here is operationally useless. *Driverlog* is a moderately difficult domain with medium-length plans, a large goal space, and real-world logistics vocabulary (drivers, trucks, packages, locations), testing whether LLM world knowledge transfers to a combinatorially harder setting. *DWR* is a difficult domain for both classical search and LLM-based recognisers, with very long plans, large goal spaces, and many interdependent objects to track across steps.

For each domain, the problems are divided into five observability degrees: [10%, 100%], in increments of 20%, of the plan length. Recall that we define the observability in goal recognition as the percentage of the plan’s actions that are observed by the recogniser. Therefore, the lower the observability degree, the harder the problem is as more goals may share the observed plan. Pereira and Meneguzzi [10] generate each problem by randomly retaining actions from the full plan with probability equal to the observability degree, running this sampling up to three times per plan and discarding duplicate observation sets; at 100% observability the recogniser always receives the full plan, so each plan yields exactly one problem, resulting in fewer problems at full observability than at partial observability.

¹OpenAI’s GPT-OSS; available via Ollama.

²Alibaba’s Qwen 3.5-9B; available via Ollama.

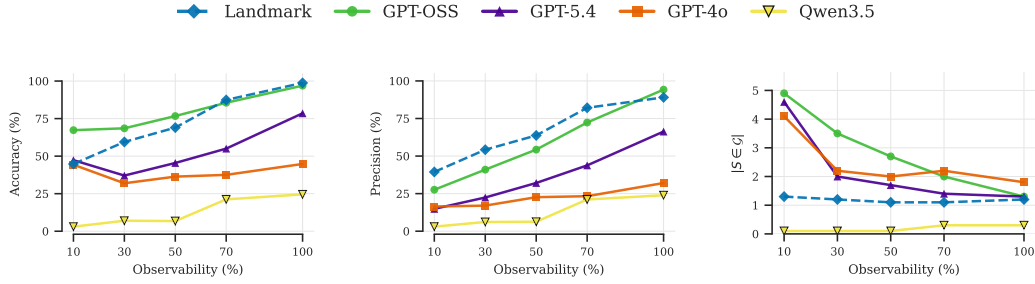


Figure 1: Average acc, prec, and spread for different observation percentages for ALL DOMAINS.

by contrast, remains effectively flat: 44.3% in accuracy at 10% observations and 44.8% at 100%, a range of less than one percentage point despite a tenfold increase in evidence. Qwen 3.5 fails almost entirely on all but the simplest domain, reaching only 24.5% at full observations across all domains.

The accuracy gap between GPT-4o and GPT-OSS is clearest on *Blocks World*, where landmark-based and GPT-4o are tied at 10% observations (44.3% and 43.5% respectively), but diverge completely thereafter: landmark-based reaches 100% at full observations, GPT-OSS reaches 96.9%, and GPT-4o never exceeds 43.5%. On *DWR*, the hardest domain, GPT-OSS reaches 92.9% at full observations, matching landmark-based exactly, while GPT-4o remains near 25% regardless of evidence. *Campus*, the simplest domain, is the exception: GPT-4o achieves 100% accuracy at all observability degrees, suggesting that world knowledge alone suffices when the goal space is small ($|\mathcal{G}| = 2$) and the vocabulary familiar. Qwen 3.5 fails even on campus at 30% and 100% observations, indicating that local inference at this scale is insufficient for reliable output formatting, not merely for reasoning.

Accuracy alone is an incomplete picture because a recogniser returning all candidate goals achieves perfect recall at the cost of zero discriminative power. Spread and precision track this tradeoff directly. We use the spread to compute the precision on each problem, dividing the number of correctly identified goals (which can be at most one) by the number of returned hypotheses. At 10% observations, GPT-OSS and GPT-4o start with similarly high spread (4.9 and 4.1 respectively, versus 1.3 for landmark-based), reflecting that both LLMs hedge broadly when evidence is sparse. Even with a higher spread, GPT-OSS achieves 27.6% precision at 10% observations due to its higher accuracy, compared to 16.4% for GPT-4o and 39.4% for landmark-based. As observations accumulate, GPT-OSS resolves its uncertainty: spread falls to 1.3 at full observations, approaching landmark-based (1.2), and precision rises to 94.2%, the best overall on full observations. GPT-4o’s precision increase for higher observability is solely due to its spread decreasing, as its accuracy remains flat. Its spread decreases only to 1.8 and accuracy remains flat at full observations, so precision reaches just 32.1%, compared to 89.1% for landmark-based and 94.2% for GPT-OSS. The precision gap between GPT-OSS and GPT-4o at full observations is thus as striking as the accuracy gap: both can retrieve the correct goal, but only GPT-OSS commits to it. On the other hand, GPT-5.4 shows a more balanced improvement in both accuracy and spread, leading to a steady increase in precision from 14.9% at 10% observations to 66.3% at 100%. Similarly to its accuracy results, Qwen 3.5’s precision is also low across all observation percentages, reaching only 23.9% at full observations. We also see that Qwen 3.5’s spread’s average is mostly below 1, due to the fact that it often fails completely and returns no candidate goals at all. Figure 1 shows these trends more clearly. We discuss the reasoning inconsistencies that lead to this kind of anomaly in the results in Section 6.

In terms of recognition time, the landmark-based baseline is by far the most efficient, with its average time ranging from 1.7 to 4.5 seconds across all domains and observation percentages. Among the LLM-based recognisers, GPT-4o is the fastest one, with the average recognition time ranging from 4.9 to 12.1 seconds, followed by GPT-5.4. Qwen 3.5 and GPT-OSS are the slowest, which is due to the fact that they are non-commercial models running on local hardware.

Regarding token usage, Table 2 shows that GPT-4o is overall the most efficient model in terms of the usage of completion tokens, while GPT-5.4 consumes less prompt tokens. GPT-4o dominates as the most token-efficient model, with a total average of around 3,000 tokens per problem, compared to around 3,300 for GPT-5.4. GPT-OSS and Qwen 3.5 are the least efficient, with 11,500 total tokens

Table 2: Token usage and cost in USD (avg. per problem) for LLMs on all domains.

Domain	O %	P	G	O	Qwen3.5			GPT-OSS			GPT-4o				GPT-5.4			
					PT	CT	TT	PT	CT	TT	PT	CT	TT	Cost	PT	CT	TT	Cost
ALL DOMAINS	10	429	14.5	2.7	2135.9	13656.4	15792.3	2003.3	6824.3	8827.6	2031.0	956.0	2987.0	0.01	2030.0	1216.4	3246.4	0.02
	30	429	14.5	7.4	2202.2	16258.8	18460.9	2155.0	10301.8	12456.7	2089.8	918.5	3008.3	0.01	2088.8	1220.2	3309.0	0.02
	50	429	14.5	11.8	2283.6	18912.6	21196.2	2211.3	10626.4	12837.6	2146.1	903.5	3049.6	0.01	2145.1	1241.6	3386.6	0.02
	70	429	14.5	16.8	2354.5	15529.1	17883.7	2273.6	11220.5	13494.0	2208.4	914.2	3122.6	0.01	2207.4	1232.5	3439.9	0.02
	100	163	14.1	21.3	2395.4	18072.7	20468.0	2331.3	7836.0	10167.4	2266.3	882.9	3149.1	0.01	2265.3	1127.8	3393.0	0.02

used for GPT-OSS, and 18,700 for Qwen 3.5. In terms of cost in the commercial models, all of them kept the average cost per problem around 0.01 and 0.02 USD. We report the complete information on the cost of commercial models in terms of the average number of input and output tokens, as well as the average cost per recognition, in Table 3 in Appendix B.

6 Discussion

Given the quantitative results, we now focus on analysing the reasoning process of LLMs when performing goal recognition, and provide some examples where we believe improvements can be made to allow LLMs to perform better. We select examples from the *Blocks-World* domain, as it is a commonly used domain in the goal recognition literature, and one that is adopted for the planning experiments in Corrêa et al.’s work [2025]. We reference figures containing the reasoning excerpts in Appendix C for optional review.

GPT-4o GPT-4o’s reasoning is consistently inconsistent with its own score assignments, a pattern that holds across all observability levels. At low observability, the model tends to anchor on the few observed actions and check which hypotheses include the affected blocks, but does so overconfidently: in Figure 3, it selects a single hypothesis despite multiple others being equally compatible with the observation, and its justification contains factual errors about the preconditions. This overconfidence is compounded by *confabulation*, where the model produces plausible-sounding justifications that directly contradict its own scores: in Figure 4 it claims no hypothesis mentions the relevant block, when several do; in Figure 5 it states it favoured hypotheses involving block P, yet the correct hypothesis does not prominently include P; and in Figure 6 the reasoning identifies discriminating facts and then collapses to a uniform distribution without explanation. At higher observability, the failure mode shifts but the underlying problem persists: in Figure 10 the reasoning names block P as relevant but the scores ignore it; in Figure 8 the model identifies a relation between blocks A and J that is not supported by the observations; and in Figure 9 the conclusion abruptly introduces blocks M and O, neither present in the observations nor in the preceding reasoning, to justify assigning maximum score to an incorrect hypothesis. Across all cases, evidence that is named in the reasoning is not enforced in the scores, and evidence that drives the scores is not grounded in the reasoning.

GPT-5.4 GPT-5.4 exhibits more sophisticated planning reasoning than GPT-4o, but this comes with its own failure modes. Unlike GPT-4o, it identifies precondition violations arising from partial observability and reasons around them rather than ignoring them: in Figure 14, it correctly infers that two of the three observed actions jointly establish $(ON\ P\ A)$, identifies the unique hypothesis containing that fact, and selects it correctly. It also reasons about hypotheses as goals that the observed sequence may be partially progressing toward, which is a principled strategy under partial observability, as illustrated in Figure 16, where it narrows to two candidates and selects the one most consistent with the observation as a partial plan. However, this same tendency to commit early becomes a liability when evidence is insufficient or the problem is complex: under full observability it selects the wrong hypothesis in Figure 16 by misreading a completed sequence as an incomplete one, it derives an entirely incorrect 10-block final stack in Figure 15 despite having all actions available, and in Figure 17 it overcommits to a single hypothesis at 30% observability despite initially narrowing correctly to a set of four candidates that included the correct goal. The net effect is higher precision when the reasoning is right, but lower accuracy when it is not.

GPT-OSS GPT-OSS displays a recurring behaviour of applying a fact-matching strategy in most problems, trying to check the actual facts present in the state resulting from the execution of the observed actions. In Figure 11, the model correctly identifies the correct hypothesis by reasoning about what are the facts present in the state following the applying of the single observed action, and

assigns to each hypothesis a score proportional to the number of matched facts. By doing so, it is able to obtain more consistent reasoning, but this behaviour still leads it to incorrect inference many times, like in the example from Figure 12. In this example, the model does its best with the provided information, and correctly identifies the hypothesis that best matches the observed actions, but since there is lacking information due to partial observability, the model interprets an intermediate state as a final one. Since in the reasoning it correctly identifies the missing information by stating that it is ignoring the preconditions being violated, it could in theory try to better distribute the score to include a larger number of matching hypotheses. Different from the previous example, where the model explicitly informed about ignoring the violated preconditions, in another problem the same model stated that since the observations contained precondition violations, it could not infer the correct intended goal, and hence assigned a uniform score to all hypotheses (Figure 13). This showcases an inconsistency in the model’s behaviour, which is expected to some degree given the nature of LLMs. Additionally, it is interesting to see how it deals with the same issue differently, considering that the prompt does not specify that the observation sequence can be a non-contiguous partial subset of the full observation sequence.

Qwen 3.5 Out of all four evaluated models, Qwen 3.5 was the worst when it comes to reasoning inconsistencies and hallucinations. For many problems across all domains and observability degrees, the model did not provide any response at all, returning a completely empty output, even while consuming tokens. In other cases, it correctly identifies a precondition violation under partial observability, but in turn simply states the sequence of actions is impossible and refuses to provide any further response, as in the example from Figure 18. In a specific problem, Qwen 3.5 provides unsound reasoning, showcasing explicit confusion and even confusing block names. It also fails to deliver a response in the required format and select a hypothesis that was not even in the hypothesis list (Figure 19). In another problem, with 30% observability, the model fails completely and generates a nonsensical response, with its only response being “You put x on table.” as shown in Figure 20. Another example of this kind of behaviour from Qwen 3.5 is shown in Figure 21, where the model provides a completely incoherent response comprised only of “[answer A]” under full observability. For both cases, the model used around 12,000 completion tokens, meaning there was reasoning effort but still no sound response. Still, there are cases where Qwen 3.5 shows reasoning capabilities and even correctly narrows down to a single and correct hypothesis under sensible reasoning, as in the example from Figure 22. In this example, even under 50% observability, the model correctly identifies the resulting state from the observed actions and narrows down to two possibly correct hypotheses. After that, it identifies an inconsistency between one of the selected hypotheses and the initial state, unchanged by the observations, and hence correctly selects the other hypothesis as the most likely correct one.

Common Patterns Despite their differences in capability and failure mode, the models examined here share a consistent set of weaknesses that together point toward a productive research agenda. All models struggle to translate stated reasoning into consistent score assignments, all exhibit overconfidence when observations are sparse, and none reliably improve their predictions as additional evidence accumulates, which is the defining discriminative demand of goal recognition. All models also display position bias consistent with ‘lost-in-the-middle’ [6]: attention degrades for hypotheses placed in the middle of the candidate list, with GPT-4o exhibiting a pronounced tendency to favour hypotheses at the first position. Yet the broader picture is not discouraging. The models collectively demonstrate that world knowledge of planning domains transfers meaningfully to zero-shot goal recognition: even without fine-tuning, all models perform non-trivially at low observability, and the progression from GPT-4o to GPT-5.4 shows that stronger general reasoning capabilities do address specific failure modes, notably precondition awareness and partial-plan reasoning. This suggests that the bottleneck is not knowledge but the ability to integrate evidence incrementally. Several targeted interventions could plausibly close the remaining gap. Restructuring the output format to elicit reasoning before scores would allow deliberate comparison of candidate goals to genuinely constrain the prediction. Training models to maintain an explicit running state representation updated action by action would directly address the state-tracking failures observed at higher observability. Improving calibration so that models express uncertainty proportional to the available evidence would reduce the precision-accuracy trade-off that characterises GPT-5.4’s behaviour. A systematic prompt sensitivity analysis varying hypothesis ordering and position would further quantify the extent of position bias and inform more robust evaluation protocols [19]. More broadly, goal recognition provides an unusually clean benchmark for the foundational planning knowledge of LLMs: the ground truth is unambiguous, the evidence is structured and incremental, and the failure modes are

precisely measurable. The plateau at low observability reveals the floor set by world-knowledge priors; the divergence in scaling behaviour across models reveals how far that foundational knowledge alone can carry recognition and where evidence integration becomes the limiting factor.

7 Related Work

The relationship between LLMs and automated planning is one of the most actively studied questions in recent AI research. Early systematic evaluation using the PlanBench benchmark shows that LLMs fail to reliably solve classical planning problems; on standard IPC domains zero-shot performance is near chance, and it collapses entirely when domain and object names are obfuscated to remove semantic cues [18, 20]. Kambhampati et al. [5] argue against using LLMs as standalone planners and propose the LLM-Modulo framework, which embeds LLMs within a loop controlled by external formal verifiers. A complementary finding is that LLMs cannot reliably verify their own planning outputs [16]. More recent benchmarking finds that frontier LLMs and reasoning models have closed much of the gap with classical planners on standard IPC domains [21, 3]; obfuscated variants continue to expose substantial performance drops, confirming that this progress rests on world-knowledge exploitation rather than symbolic search.

Chiari et al. [2] introduce GRNet, which recasts goal recognition as a deep learning classification task: a neural network trained on plan traces from a given domain learns to predict the goal from a partial observation sequence. Serina et al. [15] extend this line to online settings, where the recogniser updates its prediction incrementally as new observations arrive. These approaches achieve high accuracy and fast inference but require domain-specific training data, making them unsuitable for zero-shot evaluation across novel domains. A distinct line of work addresses goal recognition under noisy or incomplete observations using neuro-symbolic methods [12, 17]; that setting differs from ours, in which observations are assumed complete but partial. Our contribution is complementary to both lines: we ask whether world knowledge encoded in pre-trained LLMs suffices for zero-shot goal recognition, without any domain-specific training.

8 Conclusion

This paper provides the first systematic evaluation of LLMs in the task of goal recognition over PDDL benchmarks. Given the substantial challenge [20], and recently perceived progress [3], these benchmarks provide a closely related, but qualitatively different challenge to language models, and how they employ world knowledge. Our empirical experiments show that even models that approximate the performance of symbolic algorithms for planning (GPT-5.4) collapse when performing goal recognition in the same domain. These failures happen even without any obfuscation with three qualitatively distinct profiles. GPT-OSS leads all models at every observation percentage, scaling from 67.3% to 96.9% accuracy and approaching LM (98.8%) at full observations. GPT-5.4 shows partial scaling (47.3% to 78.5%). GPT-4o remains flat at 44% regardless of evidence. Qwen 3.5 fails almost entirely due to output-format failures at this model scale.

The key discriminator between profiles is not domain knowledge but the ability to integrate additional evidence: GPT-4o matches LM at 10% observations, where world-knowledge priors suffice, then diverges completely as evidence accumulates. This is the predicted failure mode from Section 3.2: world-knowledge retrieval masquerading as goal recognition. Here GPT-4o indicates that more domain knowledge acts as a disadvantage if the models do not perform reasoning. Our prompt places likelihood scores before the free-text reasoning block as a deliberate methodological control. It requires each model to commit to a distribution over goals using only its domain knowledge and the observation evidence, preventing the reasoning trace from post-hoc steering the scores toward a rationalised answer. GPT-4o’s flat accuracy curve therefore reflects a genuine failure to integrate evidence rather than an artefact of the model being steered toward prior-anchored scores by its own reasoning. Whether reversing the order, reasoning first and scoring after, would improve evidence integration for models like GPT-4o is an open question and a natural next step.

Goal recognition provides a principled benchmark for LLM reasoning: the accuracy floor at sparse observations measures the domain familiarity encoded in training, while the scaling slope over increasing observations reveals whether a model integrates evidence or merely retrieves priors. The contrast between GPT-4o (flat slope, high floor) and GPT-OSS (rising slope) makes this separation

empirically clean. Several directions remain open: (i) a mystery-domain ablation to test whether GPT-OSS’s scaling survives name obfuscation [18]; (ii) chain-of-thought prompting with reasoning placed before scores, to test whether evidence integration improves; and (iii) evaluation under noisy observations, where some observed actions are incorrect, a setting in which LLMs’ approximate reasoning may provide an advantage over classical methods that assume noiseless input.

Acknowledgments and Disclosure of Funding

References

- [1] Tom Bylander. The computational complexity of propositional strips planning. *Artificial Intelligence*, 69(1-2):165–204, 1994.
- [2] Mattia Chiari, Alfonso Emilio Gerevini, Francesco Percassi, Luca Putelli, Ivan Serina, and Matteo Olivato. Goal recognition as a deep learning task: The grnet approach. In Sven Koenig, Roni Stern, and Mauro Vallati, editors, *Proceedings of the Thirty-Third International Conference on Automated Planning and Scheduling*, pages 560–568. AAAI Press, 2023. doi: 10.1609/ICAPS.V33I1.27237.
- [3] Augusto B. Corrêa, André Grahl Pereira, and Jendrik Seipp. The 2025 planning performance of frontier large language models. *CoRR*, abs/2511.09378, 2025. doi: 10.48550/ARXIV.2511.09378.
- [4] J Hoffmann, J Porteous, and L Sebastia. Ordered landmarks in planning. *Journal of Artificial Intelligence Research*, 22(1):215–278, 2004. doi: 10.1613/jair.1492.
- [5] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Kaya Stechly, Mudit Verma, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. LLMs can’t plan, but can help planning in LLM-modulo frameworks. In *Proceedings of the Forty-First International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2402.01817>.
- [6] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638. URL <https://aclanthology.org/2024.tacl-1.9/>.
- [7] Felipe Meneguzzi and Ramon Fraga Pereira. A survey on goal recognition as planning. In Zhi-Hua Zhou, editor, *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, pages 4524–4532, 2021. doi: 10.24963/ijcai.2021/616.
- [8] Reuth Mirsky, Sarah Keren, and Christopher Geib. *Introduction to Symbolic Plan and Goal Recognition*, volume 16. Morgan & Claypool Publishers, 2021.
- [9] Bernhard Nebel. On the compilability and expressive power of propositional planning formalisms. *Journal of Artificial Intelligence Research*, 12:271–315, 2000.
- [10] Ramon Fraga Pereira and Felipe Meneguzzi. Goal and plan recognition datasets using classical planning domains, 2017.
- [11] Ramon Fraga Pereira, Nir Oren, and Felipe Meneguzzi. Landmark-based approaches for goal recognition as planning. *Artificial Intelligence*, 279:103217, 2020. doi: 10.1016/j.artint.2019.103217. URL <https://linkinghub.elsevier.com/retrieve/pii/S0004370219300013>.
- [12] Ramon Fraga Pereira, Francesco Fuggitti, Felipe Meneguzzi, and Giuseppe De Giacomo. Temporally extended goal recognition in fully observable non-deterministic domain models: Temporally extended goal recognition in fond planning. *Applied Intelligence*, December 2023. ISSN 1573-7497. doi: 10.1007/s10489-023-05087-1.
- [13] Miquel Ramírez and Hector Geffner. Plan recognition as planning. In Craig Boutilier, editor, *Proceedings of the 21st International Joint Conference on Artificial Intelligence*, pages 1778–1783, 2009. URL <http://ijcai.org/Proceedings/09/Papers/296.pdf>.

- [14] Miquel Ram  rez and Hector Geffner. Probabilistic plan recognition using off-the-shelf classical planners. In *AAAI Conference on Artificial Intelligence*, pages 1121–1126, 2010. URL <http://dblp.org/rec/conf/aaai/RamirezG10>.
- [15] Lorenzo Serina, Mattia Chiari, Alfonso Emilio Gerevini, Luca Putelli, and Ivan Serina. Towards efficient online goal recognition through deep learning. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*, pages 1895–1903. International Foundation for Autonomous Agents and Multiagent Systems, 2025. ISBN 9798400714269. URL <https://iris.unibs.it/bitstream/11379/628225/1/3709347.3743826.pdf>.
- [16] Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. On the self-verification limitations of large language models on reasoning and planning tasks. In *Proceedings of the Thirteenth International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2402.11686>.
- [17] Douglas Tesch, Leonardo Amado, and Felipe Meneguzzi. Real-time goal recognition using approximations in Euclidean space. In *27th European Conference on Artificial Intelligence (ECAI)*, 2024. URL <http://www.meneguzzi.eu/felipe/pubs/ecai-vector-gr-2024.pdf>.
- [18] Karthik Valmeekam, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. Large language models still can’t plan (a benchmark for LLMs on planning and reasoning about change). In *NeurIPS 2022 Workshop on Foundation Models for Decision Making*, 2022. URL <https://arxiv.org/abs/2206.10498>.
- [19] Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. Planbench: an extensible benchmark for evaluating large language models on planning and reasoning about change. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS ’23, Red Hook, NY, USA, 2023. Curran Associates Inc.
- [20] Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. On the planning abilities of large language models – a critical investigation. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2302.06706>.
- [21] Karthik Valmeekam, Kaya Stechly, Atharva Gundawar, and Subbarao Kambhampati. A systematic evaluation of the planning and scheduling abilities of the reasoning model o1. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=FkKBxpOFhR>.
- [22] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

A Prompt Template

```
You are an expert at goal recognition in AI planning using planning domain descriptions provided in PDDL format.
You will be provided with the following:

- Planning domain description in PDDL. This contains the definition of the possible types of objects and the
  description of the actions that can be performed in the environment. Each action is comprised by the following:
  - A name
  - A list of parameters (with their types)
  - A list of precondition represented as a logical AND of facts, meaning facts that must hold in the state for the
    action to be applicable
  - A list of effects represented as a logical AND of facts, meaning facts that will be added to the state or
    removed from the state if they are preceded by the reserved word "not" after applying the action to the state
- Problem template, which is the PDDL description of the corresponding planning problem with the initial state and a
  placeholder for the goal state, comprising the following:
  - A list of objects in the problem with their types
  - An initial state represented as a simple list of facts separated by new line characters that hold in the initial
    state of the problem
  - A placeholder for the goal state which would be represented as a logical AND of facts, meaning facts that must
    hold in the final and goal state of the problem
  - A list of goal hypotheses where one of them is the correct goal. In this list each goal is a list of facts. Each
    fact in a goal is separated by comma "," characters. Each goal hypothesis in the list is separated by new line
    characters.
  - A list of observed actions that were performed by the agent. This is a simple list of executed actions separated by
    new line characters.

Your task is to analyze the problem provided to you and determine which goals among the goal hypotheses are the most
likely to be the correct ones, given the observed actions of an agent.
For each goal hypothesis, assign a likelihood score (real number between 0 and 1) that represents how likely it is
that the observed agent is pursuing that goal, based on the actions observed. This score MUST be normalized, so the
sum of the likelihood scores of all goal hypotheses must be equal to 1. After assigning a likelihood score to each
goal hypothesis, determine the set of goals which are tied with the highest likelihood score and provide an
explanation of the reasoning process that led you to assign those likelihood scores and determine that set of most
likely goals.
The output should be the list of goals along with their likelihood scores, the set of goals which are tied with the
highest likelihood score, and after that an explanation of the reasoning process that led to the final answer. The
format of the output should be exactly as the one specified below the "## Output Format" section of this prompt,
without any additional text or formatting.

## Domain
{problem.domain}

## Problem Template
{problem.template}

## Goal Hypotheses
{goals_text}

## Observed Actions (in order)
{obs_text}

## Output Format (follow this format exactly, without any additional text or formatting):

Hyps:
Hyp: <goal hypothesis> | Score: <likelihood score between 0 and 1>

Most Likely Goals:
- <goal hypothesis 1>
- <goal hypothesis 2>
...

Reasoning:
<explanation of the reasoning process that led to the final answer>
```

Figure 2: Prompt template used for zero-shot goal recognition. Placeholders {problem.domain}, {problem.template}, {goals_text}, and {obs_text} are filled at runtime with the PDDL domain, problem template, goal hypotheses, and observed action sequence respectively.

B Token Usage and Cost

Table 3: Token usage and cost in USD (avg. per problem) for LLMs on all problems.

Domain	$\mathcal{O}$ %	P	G	C	Qwen3.5			GPT-OSS			GPT-4o				GPT-5.4			
					PT	CT	TT	PT	CT	TT	PT	CT	TT	Cost	PT	CT	TT	Cost
BLOCKS WORLD	10	246	20.3	1.8	1862.6	14603.6	16466.2	1569.6	7441.7	9011.3	1766.5	1107.5	2874.0	0.02	1765.5	1363.9	3129.4	0.02
	30	246	20.3	4.9	1903.4	15119.8	17023.2	1865.0	11664.3	13529.3	1796.0	1060.3	2856.3	0.02	1795.0	1329.0	3124.0	0.02
	50	246	20.3	7.7	1933.6	20369.4	22303.0	1890.7	12266.6	14157.2	1821.7	1042.2	2863.9	0.01	1820.7	1357.7	3178.4	0.02
	70	246	20.3	11.1	1972.0	14926.9	16898.9	1923.3	12590.8	14514.1	1854.3	1065.7	2920.0	0.02	1853.3	1335.5	3188.8	0.02
	100	92	20.3	14.5	1995.2	18649.6	20644.8	1940.1	8975.4	10915.5	1871.1	1035.4	2906.5	0.02	1870.1	1244.7	3114.8	0.02
CAMPUS	10	15	2.0	1.0	2606.9	10672.1	13279.1	2492.5	2474.7	4967.2	2432.5	350.1	2782.7	0.01	2431.5	469.2	2900.7	0.01
	30	15	2.0	2.0	2621.7	10285.3	12907.0	2505.7	2994.6	5500.3	2445.7	373.8	2819.5	0.01	2444.7	538.3	2983.1	0.01
	50	15	2.0	3.0	2633.9	12413.7	15047.5	2516.9	2644.5	5161.4	2456.9	376.4	2833.3	0.01	2455.9	579.2	3035.1	0.01
	70	15	2.0	4.5	2655.6	10565.6	13221.2	2537.1	2651.4	5188.5	2477.1	367.5	2844.7	0.01	2476.1	606.4	3082.5	0.02
	100	15	2.0	5.4	2667.3	11792.2	14459.5	2548.9	2389.1	4937.9	2488.9	432.3	2921.1	0.01	2487.9	607.4	3095.3	0.02
DRIVERLOG	10	84	7.1	2.6	2768.2	13239.7	16007.9	2726.0	5542.3	8268.3	2666.0	850.3	3516.3	0.02	2665.0	1118.9	3783.9	0.02
	30	84	7.1	7.0	2825.6	13143.2	15968.8	2795.2	7671.5	10466.8	2735.2	824.5	3559.7	0.02	2734.2	1161.8	3896.0	0.02
	50	84	7.1	11.2	2989.7	17850.1	20839.8	2862.6	7111.2	9973.8	2802.6	803.1	3605.8	0.02	2801.6	1164.8	3966.4	0.02
	70	84	7.1	15.6	3068.1	18628.5	21696.7	2933.7	7235.7	10169.4	2873.7	813.8	3687.5	0.02	2872.7	1166.8	4039.5	0.02
	100	28	7.1	21.7	3014.4	12628.0	15642.4	3030.4	4676.5	7706.9	2970.4	800.1	3770.5	0.02	2969.4	1115.2	4084.6	0.02
DWR	10	84	7.3	5.7	2219.6	11832.1	14051.7	2158.8	7508.4	9667.2	2098.8	726.3	2825.1	0.01	2097.8	1015.3	3113.1	0.02
	30	84	7.3	16.0	2378.7	23776.6	26155.2	2301.2	10246.7	12547.9	2241.2	694.5	2935.7	0.01	2240.2	1082.0	3322.2	0.02
	50	84	7.3	26.2	2539.8	16869.6	19409.4	2444.1	10763.6	13207.7	2384.1	692.0	3076.1	0.01	2383.1	1096.5	3479.6	0.02
	70	84	7.3	36.9	2707.5	15079.9	17787.4	2592.2	12722.2	15314.4	2532.2	668.6	3200.8	0.01	2531.2	1108.4	3639.5	0.02
	100	28	7.3	51.9	2945.4	24986.2	27931.5	2801.3	10169.8	12971.0	2741.3	705.9	3447.2	0.01	2740.3	1034.8	3775.1	0.02
ALL DOMAINS	10	429	14.5	2.7	2135.9	13656.4	15792.3	2003.3	6824.3	8827.6	2031.0	956.0	2987.0	0.01	2030.0	1216.4	3246.4	0.02
	30	429	14.5	7.4	2202.2	16258.8	18460.9	2155.0	10301.8	12456.7	2089.8	918.5	3008.3	0.01	2088.8	1220.2	3309.0	0.02
	50	429	14.5	11.8	2283.6	18912.6	21196.2	2211.3	10626.4	12837.6	2146.1	903.5	3049.6	0.01	2145.1	1241.6	3386.6	0.02
	70	429	14.5	16.8	2354.5	15529.1	17883.7	2273.6	11220.5	13494.0	2208.4	914.2	3122.6	0.01	2207.4	1232.5	3439.9	0.02
	100	163	14.1	21.3	2395.4	18072.7	20468.0	2331.3	7836.0	10167.4	2266.3	882.9	3149.1	0.01	2265.3	1127.8	3393.0	0.02

C Reasoning Excerpts

The single observed action is (PICK-UP O), which means that the agent's immediate intent was to grasp the block 'O' from the table. This operation directly affects the precondition (ONTABLE O), removing it and causing the holding condition for 'O' to be true while reducing the hand being empty status to false. Evaluating the hypotheses, the most likely goal should involve subsequent actions that make sense in the wake of the current action, in this case removing the restrictions upon (CLEAR M), (ONTABLE W), (ON M O), (ON O W) needed for further stacking or ordering, particularly those involving 'O'. The hypothesis (CLEAR M), (ONTABLE W), (ON M O), (ON O W) suggests that the action of picking up 'O' from the table is a first logical step, supporting future moves to potentially create this arrangement, including placing 'M' on 'O' or 'W', indicating it's a highly plausible sequence of intent, hence scoring highest.

Figure 3: GPT-4o's reasoning for a problem with 10% observability in the *Blocks World* domain. The model correctly identifies the most likely goal hypothesis based on the single observed action.

The observed action, (PUT-DOWN D), directly affects the placement of block D. However, none of the goal hypotheses explicitly mention the block D being on the table or in any specific configuration that would uniquely indicate the pursuit of such a goal given only this step so far. Since no unique distinguishing action was observed, we assume equal likelihoods for each goal hypothesis considering there is just one observed action and no specific additional evidence favoring any single hypothesis over others. Therefore, all hypotheses are equally probable under the conditions of the given data.

Figure 4: GPT-4o's reasoning for a problem with 10% observability in the *Blocks World* domain. The model reasons that the single observed action does not provide enough evidence to discriminate among the candidate goals, leading it to assign equal likelihoods to all hypotheses. This results in a high spread and low precision, even though the correct intended goal is included in the returned set.

The observed action is "PICK-UP P", which indicates that the agent's first move was to pick up block P from its initial position on the table. This move directly affects hypotheses where block P is involved in transitioning from its initial state. Hypotheses that reasonably involve moving P or requiring P as a part of the stack tend to have higher scores. The action does not strongly discriminate the specific arrangement of blocks post-initiation but highlights the immediate importance of P. Hypotheses that do not involve P or the area directly around P were scored lower. Consequently, the goals most compatible with this initial action involve P in a stack or clear position, suggesting various sequential possibilities, indicated by the variety of top-scoring hypotheses. Normalizing these based on relativity to P's starting state resulted in the final scores.

Figure 5: GPT-4o's reasoning for a problem with 10% observability in the *Blocks World* domain. The model does narrow down to 8 selected hypotheses from 21 available. However, it is wrong in this case, as the true goal does not prominently involve block P.

The initial state and the observed action suggest that the agent's first move was to stack block O onto block R. This action is executed from a state where O is clear and on the table, and R is also clear. After performing STACK O R, both blocks O and R will no longer be clear due to being part of a stack, and the agent's hand will be empty again.

From the goal hypotheses, any goal that includes (ON O R) or has an arrangement that could have O directly on R could possibly have been targeted by this action. However, as the only observed action is STACK O R, it doesn't provide enough information to distinctly prefer one goal over another as it could be a step towards many different configurations described in the hypotheses.

Thus, every goal hypothesis receives an equal likelihood score, reflecting the uncertainty given the lack of more distinctive actions or evidence to suggest a particular goal over others.

Figure 6: GPT-4o's reasoning for a problem with 10% observability in the *Blocks World* domain. The model reasons about a possible way to select a smaller set of hypotheses but ends up selecting all of them.

The observed actions are (PICK-UP A), (STACK A S), and (STACK C A). The sequence of actions begins with picking up block A and placing it on block S, followed by placing block C on top of block A. This indicates a build-up with C at the top of the stack involving A and S. Among the goal hypotheses, (CLEAR C), (ONTABLE H), (ON C A), (ON A S), (ON S H) directly aligns with this sequence of observed actions since it results in a stack of C on A, and A on S, consistent with the observed agent behavior. While there may be other hypotheses that could eventually end up with A, S, and C in some form, none align specifically with the stacking pattern of the observed actions as closely as this hypothesis. Other hypotheses have lower scores due to their lack of direct correlation with the specific order and purpose of the observed stack operations.

Figure 7: GPT-4o's reasoning for a problem with 30% observability in the *Blocks World* domain. The model correctly identifies the most likely goal hypothesis based on the observed sequence of actions, which clearly indicates a stacking pattern involving blocks A, S, and C.

The observed actions indicate an intent to rearrange the blocks in a manner that correlates most closely with two of the hypotheses. The actions include several unstacking and stacking operations where blocks are rearranged to achieve a very specific configuration pattern. Notably:

1. The initial unstacking of block B from G and subsequent stacking onto J suggests an effort to create a sub-structure involving (ON A J), reflective of moving towards a goal involving stacks initiated by A/J.
2. Actions such as stacking K on F and G suggest an intent to form a sequence that can end with (ON F E), (ON E G), corresponding directly with hypothesis patterns where F is involved in the base transitions of the structure.
3. Moreover, the repeated adjustments involving D and J imply significant reordering involving these blocks, in line with the corresponding hypotheses.
4. Finally, observed tasks like stacking A on J directly align with a sequence that follows the thematic transition in two of the hypothesized stack orders, indicating a prioritized sequence in these constructs.

The two hypotheses that arise as most likely are those with a focus on initial D/J/A to H projects transitioning into wider stack compositions, represented in the final, observed movements and configurations occurring drastically at the functional goal end.

Figure 8: GPT-4o's reasoning for a problem with 70% observability in the *Blocks World* domain. The model fails to reason on a more complex problem with a larger number of objects involved and narrows down to 2 wrong hypotheses.

To evaluate which goal hypothesis is most likely, we examine how the sequence of actions aligns with each goal state's required configuration. The sequence of actions observed is such that the final goal state must have block T on block H, block A on block W, and block R on the table.

1. The UNSTACK T W and STACK T M operations suggest T is involved in the final configuration.
2. Subsequently, UNSTACK R A, PUT-DOWN R, PICK-UP A, and STACK A W indicate that R must be on the table while A is on W.
3. Finally, STACK T H places T on H, making H a supporting block, implying H must support other blocks.

Given the executed actions, the only hypothesis producing such a state is (CLEAR M),(ONTABLE W),(ON M O),(ON O W). This configuration would have M and O stacked together, with M clear and W clear, and the rest of the blocks not violating any constraints from actions. The entire configuration needed to execute the observed actions fits perfectly with this particular goal hypothesis. Hence, this goal hypothesis is assigned a likelihood score of 1.0 and is selected as the sole most likely goal.

Figure 9: GPT-4o's reasoning for a problem with 70% observability in the *Blocks World* domain. The model wrongly selects the goal hypothesis (CLEAR M),(ONTABLE W),(ON M O),(ON O W) as the most likely one, based on the observed sequence of actions. However, the true goal does not prominently involve block M or O, leading to an incorrect prediction based on a hallucination.

The observed actions (UNSTACK R P), (PICK-UP P), and (PICK-UP R) indicate a focus on manipulating blocks R and P. The unstacking of R from P and then picking up these blocks suggests that the goal involves clearing the top of block R. Among the goal hypotheses, the goal (CLEAR R),(ONTABLE W),(ON R A),(ON A W) implies such a manipulation, as it requires R to be moved, possibly involving unstacking operations and ensuring R is clear. This hypothesis aligns with the observed actions, which seemed to clear R and reposition it, making it the most likely goal. Other hypotheses receive lower scores due to less alignment with the observed actions, either involving different block arrangements or predicting different sequencing of block manipulations.

Figure 10: GPT-4o's reasoning for a problem with 30% observability in the *Blocks World* domain. The model fails to correctly identify the most likely goal due to limited observability even though there were hypotheses involving both R and P.

The observed action is a single STACK D E. To be applicable, the agent must have been holding block D and have block E clear. This implies that prior to the action D was on some block and had been unstacked, while E was clear. After the action, the resulting facts are: CLEAR D, ON D E, HANDEMPY, and NOT CLEAR E. For each goal hypothesis we counted how many of its required facts are satisfied in this resulting state. The counts were:

Hypothesis 19: 3 facts satisfied (CLEAR W, ONTABLE E, ON D E)
 All other hypotheses satisfied at most 2 facts, some only 1 or 0.
 Because the action directly contributes to satisfying ON D E (present only in hypotheses 12 and 19) and also contributes to CLEAR W and ONTABLE E (present in some), hypothesis 19 attains the highest number of satisfied facts. We use a simple proportional likelihood model: the likelihood of each hypothesis is the number of satisfied facts divided by the total number of satisfied facts across all hypotheses, ensuring the scores sum to 1. This gives hypothesis 19 a normalized score of $3/35 = 0.0857$, which is higher than any other hypothesis. Hence hypothesis 19 is the most likely goal.

Figure 11: GPT-OSS's reasoning for a problem with 10% observability in the *Blocks World* domain. The model correctly applies its fact-matching strategy to identify the most likely goal.

I evaluated each goal hypothesis by checking how many of its facts are present in the state after executing the observed actions (ignoring precondition conflicts). For each goal I computed a match ratio = (# of matched facts)/(total facts in the hypothesis). These ratios were then normalized so that all scores sum to 1. The goal with the highest ratio was the one where all its required facts were satisfied in the final state: (CLEAR A),(ONTABLE H),(ON A S),(ON S H). Since it achieved a perfect match (4/4) this hypothesis received the highest likelihood score of 0.1091, making it the single most likely goal among those provided. The remaining hypotheses received lower scores proportionally to how many of their goal facts matched the final state.

Figure 12: GPT-OSS's reasoning for a problem with 10% observability in the *Blocks World* domain. The model fails in identifying the correct goal by considering an intermediate state instead of the final one, leading it to select an incorrect hypothesis.

All 21 goal hypotheses are treated as equally admissible because the observed action sequence is inconsistent with the domain pre-conditions, making it impossible to determine which goal the agent actually pursues. Since none of the goals can be ruled out based on the observations, each hypothesis receives an equal likelihood of $1/21 = 0.047619$, and therefore all goals tie as the most likely.

Figure 13: GPT-OSS's reasoning for a problem with 30% observability in the *Blocks World* domain. The model assigns a uniform likelihood to all hypotheses due to the observed action sequence being inconsistent with the domain preconditions, resulting in a failure to discriminate among the candidate goals.

Initial state: A is on C, D is on A, R is on P, and O, E, W are on the table and clear; hand is empty.

Observed actions:

1. UNSTACK A C - applicable, agent holds A, C becomes clear.
2. PICK-UP P - not applicable immediately after action 1 in strict Blocks World semantics, because after unstacking A the hand is not empty.

Still, the two key structural intentions revealed are:

- the agent removes A from C, freeing A from below;
- the agent then manipulates P and finally

3. STACK P A - which places P on A.

The final observed relation ON P A is highly diagnostic. Among all hypotheses, only hypothesis 20 contains (ON P A). It also requires (CLEAR P) and (ONTABLE W) and (ON A W). After stacking P on A, P is indeed clear, matching that goal component. This makes hypothesis 20 uniquely aligned with the observation.

Figure 14: GPT-5.4’s reasoning for a problem with 30% observability in the *Blocks World* domain. The model correctly infers the agent’s goal despite the observed action sequence violating the domain preconditions, demonstrating a more robust reasoning process compared to GPT-OSS.

Applying the observed actions from the initial state yields the following final tower structure over the relevant blocks: (ON J G), (ON H J), (ON E H), (ON A E), (ON I A), (ON F I), (ON D F), (ON B D), (ON C B), with G on the table. This exactly corresponds to the chain C-B-D-F-I-A-E-H-J-G when read from top to bottom.

The observations strongly show the agent systematically dismantling the initial towers and rebuilding a single tower... Several actions are redundant undo/redo actions, such as stacking and unstacking e on g, j on e, h on j, and d on f, but they do not change the final state reached after the full observation sequence. The model concludes uniquely that hypothesis 5 describes this final structure.

Figure 15: GPT-5.4’s reasoning for a problem with full observability in the *Blocks World* domain. The model fails to identify the correct goal, missing the complete ordering sequence by switching one block out of place, showcasing its difficulty in handling complex problems with many objects and long observation sequences, similar to GPT-OSS.

Initial state: U, T, R, K, C are clear and on the table; S is on A, A is on H, and H is on the table; hand is empty.

Observed actions: 1. PICK-UP U, 2. STACK U T, 3. PICK-UP C, 4. STACK C U

These actions build the tower C on U on T, yielding a state where ON U T and ON C U are true, T is on the table, and C is clear.

The observations match hypothesis 8 exactly: (CLEAR C), (ONTABLE T), (ON C U), (ON U T). After the four observed actions, every fact in that goal already holds, so this is a very strong candidate.

Hypothesis 20 is also strongly supported because it contains the achieved substructure (CLEAR C), (ONTABLE T), (ON C U), but requires extending the tower to ON U R and ON R T. This is compatible with the observation pattern: the agent may be assembling part of a larger final tower from the top downward/subtower-first...

Thus only hypotheses 8 and 20 receive nonzero probability. Hypothesis 20 is assigned the highest likelihood because the observations look like a purposeful partial construction toward a larger stack rather than necessarily stopping immediately once hypothesis 8 is reached.

Figure 16: GPT-5.4’s reasoning for a problem with full observability in the *Blocks World* domain. The model fails to identify the correct goal, assuming that the observations are on the path to reach the selected hypotheses instead of considering one that is already achieved.

Initial state: O, E, W are clear and on the table; R is on P and clear; D is on A, A is on C, and C is on the table; hand is empty.

Observed action 1 is (STACK O W). For this to be applicable, the agent must already be holding O and W must be clear. W is clear initially, so the observation strongly indicates that immediately before the observed trace the agent had picked up O. After this action, an achieved relation is (ON O W). This fact appears only in hypotheses 5, 6, 14, and 16, so those goals are supported by the first observation. All other hypotheses are inconsistent with the only positive structural fact created by the first observed action.

Observed action 2 is (UNSTACK R P). This action is applicable in the initial state after action 1, because R is on P, clear, and the hand is empty after stacking O on W. Its effect removes (ON R P), makes the agent hold R, and makes P clear. This is useful when the goal needs R moved elsewhere, or when P must become clear.

Among the goals supported by action 1:

- Hypothesis 5: needs (ON R O) and (ON O W). Since O is already on W, unstacking R from P is a sensible next step toward stacking R on O.
- Hypothesis 6: needs (ON R O) and (ON O W). Same support as hypothesis 5.
- Hypothesis 16: needs (ON C O), (ON O W), (ON W E), (ON E R). This is a longer tower involving O on W, but unstacking R from P can also be relevant because R is the intended base for E.
- Hypothesis 14: needs (ON P O), (ON O W), (ON W E), (ON E R). Here the second action is especially informative: unstacking R from P directly clears P, which is necessary if P is later to be stacked onto O. It also starts relocating R so E can be placed on R. Thus both observations align well with this hypothesis.

There is one additional hypothesis strongly supported by the second observation:

- Hypothesis 20: (CLEAR P), (ONTABLE W), (ON P A), (ON A W). Unstacking R from P achieves (CLEAR P) directly, which is one of the goal facts. Also, to build A on W and P on A, W must remain on the table and be clear enough to receive A; the earlier stack O on W is not itself a goal fact, but it may be an intermediate step that would later need to be undone, making this somewhat less cleanly aligned than the O-on-W goals. Still, the explicit achievement of (CLEAR P) by the second observed action makes this hypothesis very plausible.

Comparing supported hypotheses, hypothesis 20 is the strongest because the second observed action directly achieves one of its required goal facts, whereas for hypotheses 5, 6, 14, 16, and 21 the actions are mostly preparatory except that action 1 achieves (ON O W). Among those, hypotheses 5, 6, 16, and 21 receive some support, but less than hypothesis 20. The remaining hypotheses are given zero because they are not supported by the observed key effects and often conflict with them, especially the creation of (ON O W) or the use of R and P in the second action.

The scores are normalized to sum to 1, with the highest likelihood assigned uniquely to hypothesis 20.

Figure 17: GPT-5.4’s reasoning for a problem with 30% observability in the *Blocks World* domain. The model fails to identify the correct goal, overcommitting to a single hypothesis with lacking information.

The sequence is invalid. After (pick-up E), the robot’s hand holds E, so handempty is false. The action (pick-up C) requires handempty to be true. Therefore, you cannot pick up C while holding E. This makes the sequence impossible to execute as described.

Figure 18: Qwen 3.5’s reasoning for a problem with 10% observability in the *Blocks World* domain. The model identifies the precondition violation but refuses to continue from there.

Hypothesis 1: ‘(ONTABLE B)(ON S A)(CLEAR D)(ONTABLE D)(ON S T)’

Based on the provided text, there is significant confusion regarding the status of ‘(ON S T)’ in Hypothesis 1 and whether ‘A’ is ‘T’. The text “Wait ‘(ON S T)’ is in Hypothesis 1.” suggests that Hypothesis 1 contains ‘(ON S T)’. Additionally, the text “Wait ‘(ONTABLE D)’ is in Hypothesis 1.” and “If both ‘(ONTABLE D)’ and ‘(ON S T)’ are in Hyp 1.” implies that both are likely true. The hypothesis ‘(ONTABLE B)(ON S A)(CLEAR D)(ONTABLE D)(ON S T)’ contains both ‘(ONTABLE D)’ and ‘(ON S T)’. The fact that the text explicitly states “Wait ‘(ON S T)’ is in Hypothesis 1” and that this is a statement of fact within the provided logic, supports the inclusion of ‘(ON S T)’ in Hypothesis 1. The text “If ‘A’ is ‘T’” might be a red herring or a condition that is assumed. Given the conflicting statements, the hypothesis that contains ‘(ON S T)’ and ‘(ONTABLE D)’ is Hypothesis 1, as described in the text. Therefore, the most consistent hypothesis given the explicit “Wait” statements is Hypothesis 1.

Hypothesis 1: ‘(ONTABLE B)(ON S A)(CLEAR D)(ONTABLE D)(ON S T)’

Figure 19: Qwen 3.5’s reasoning for a problem with 10% observability in the *Blocks World* domain. The model generates unsound reasoning, generates output outside the required format, and selects a hypothesis that was not even among the provided goal hypotheses, showcasing a complete failure in understanding the task and the provided information.

You put x on table.

Figure 20: Qwen 3.5’s reasoning for a problem with 30% observability in the *Blocks World* domain. The model fails completely and generates a nonsensical response.

[:answer A]

Figure 21: Qwen 3.5’s reasoning for a problem with 30% observability in the *Blocks World* domain. The model fails completely and generates a nonsensical response.

After the first two actions (UNSTACK R P and PUT-DOWN R), the block R is placed on the table, and P becomes clear. The next two actions (PICK-UP E and STACK E R) result in E being stacked on R. This establishes the state where (ON E R) holds and R is on the table (ONTABLE R).

Among the goal hypotheses, only Hypothesis 14 and Hypothesis 16 contain the facts (ON E R) and (ONTABLE R). Hypothesis 16 requires (CLEAR C) to be true and (ON C O) to hold. However, block C is currently under block A (since (ON A C) is in the initial state and A has not been moved), implying (CLEAR C) is false. Thus, Hypothesis 16 is inconsistent with the current state and requires moving C (supported by A), which is not implied by the actions. Hypothesis 14 requires (CLEAR P) and (ON P O). (CLEAR P) is true after action 1. Since Hypothesis 14 is the only hypothesis compatible with the derived stack E on R, it is the most likely goal.

Figure 22: Qwen 3.5’s reasoning for a problem with 50% observability in the *Blocks World* domain. The model correctly identifies the most likely goal hypothesis based on the observed actions and state changes.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The claims made in the abstract and introduction are backed by the results in the paper, in Sections 5 and 6.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: The paper discusses the limitations of the work in the conclusion section (Section 8).

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper provides detailed instructions and information necessary to reproduce the main experimental results. The landmark-based method used as baseline is available in public repository, and the prompt and models used in the LLM experiments are fully disclaimed.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The paper provides detailed instructions and information necessary to reproduce the main experimental results. Moreover, the code is made available as compressed file along the submission. The code for the landmark-based baseline approach is available in public repository.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Since no training is performed, only the usage of off-the-shelf LLMs, the prompt template fully disclosed and the benchmark dataset disclosed and available in public repository, yes.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The paper does not include multiple runs of the same experiment, so error bars are not applicable.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper provides detailed information about the compute resources used for the experiments, including the type of CPU and GPU, memory, and operating system.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: the work conducted in this research has in no form violated the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: No societal impacts are expected from the work performed in this research.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper poses no such risks. All data made available to models are benchmarking problems that are already publicly available.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper explicitly mentions the models used and the companies that created them.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.

- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not release new assets. It just benchmarks existing models on an existing dataset.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [\[Yes\]](#)

Justification: The main goal of the paper is benchmarking LLMs in a specific problem setting and that is disclosed throughout the paper.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.